COE305 MACHINE LEARNING

Final Project Report

Deadline: 04-01-2026

Team Members:

Student ID	Student Name	Section (as per OIS)
2309111017	Berke DURDU	SECTION 1
2309111041	Kerem KOÇAK	SECTION 1
2309111056	Gabriel KESLER	SECTION 1

Credit Score Classification

Abstract

This project aims to classify individual credit scores into three categories: Good, Standard, and Poor. Starting with 100,000 raw records from Kaggle, we performed rigorous cleaning and outlier handling, resulting in a high-quality dataset of 73,709 samples. We evaluated six machine learning algorithms, comparing Non-Ensemble (Logistic Regression, Decision Tree, KNN) against Ensemble methods (Random Forest, Gradient Boosting, XGBoost).

The results confirmed that tree-based ensemble models significantly outperform linear baselines. The Tuned Random Forest Classifier achieved the highest performance with an Accuracy of 78.89% and F1-Score of 78.92%. Feature importance analysis identified 'Outstanding Debt' and 'Credit Mix' as the most critical predictors. The study concludes that ensemble learning is the most effective approach for handling the non-linear complexities of credit risk assessment.

Introduction

- **Background:** A global finance company has collected extensive customer credit data to determine creditworthiness.

- **Motivation:** The manual assignment of customers to credit score brackets is slow and costly. Machine learning offers a way to automate this with high precision.

- **Societal / Practical Relevance:** Automating this process improves operational efficiency, ensures objective consistency, and allows financial institutions to manage risk more effectively.

Problem Statement

The Prediction / Classification Task: The objective is to build a multi-class classification model that predicts customer creditworthiness into three brackets: Good, Standard and Poor.

The model utilizes financial features to automate this decision-making process with high accuracy.

Why the Problem is Important? Accurate credit scoring is vital for risk mitigation and operational efficiency. It allows financial institutions to minimize loan defaults by identifying high-risk clients while automating approvals for reliable customers, ensuring a consistent and fast workflow.

Limitations of Existing Solutions: Traditional manual reviews are slow, costly, and prone to human bias and error. Furthermore, simple rule-based systems often fail to capture the complex, non-linear patterns in financial data that machine learning models can easily identify.

Objectives

Develop a Robust Pipeline: To implement a comprehensive machine learning pipeline that effectively handles data cleaning, outlier detection, and feature engineering for noisy financial data.

Model Comparison: To evaluate and compare the performance of baseline models (Logistic Regression, Decision Tree, KNN) against advanced ensemble methods (Random Forest, Gradient Boosting, XGBoost) in classifying credit scores.

Hyperparameter Optimization: To maximize model performance (specifically Accuracy and Weighted F1-Score) by fine-tuning hyperparameters using RandomizedSearchCV.

Explainability: To identify the key financial drivers of creditworthiness using Feature Importance.

Dataset Description

- **Dataset Source:** Kaggle
- **Data Type:** Structured
- **Number of Samples:** 73709
- **Number of Features:** 25 (Target variable included)
- **Target Variable:** Credit_Score
- **Link for the Dataset:** https://www.kaggle.com/datasets/parisrohan/credit-score-classification

DATA PREPROCESSING & EDA

Data Pre-processing

- **Handling Missing Values:** Numerical missing values were imputed with the median while categorical gaps were filled with the mode.

```
# 3. Missing Value Handling
numeric_cols = df_cleaned.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    df_cleaned[col].fillna(df_cleaned[col].median(), inplace=True)

categorical_cols = df_cleaned.select_dtypes(include=['object']).columns
for col in categorical_cols:
    mode_val = df_cleaned[col].mode()[0] if not df_cleaned[col].mode().empty else 'Unknown'
    df_cleaned[col].fillna(mode_val, inplace=True)
```

- **Outlier Detection & Treatment:** We found some extreme values in the financial columns that didn't look realistic. To fix this without deleting too much data, we used the IQR method to clip these outliers into a more normal range.

```
# 4. Outlier Handling
numeric_features = df_cleaned.select_dtypes(include=[np.number]).columns
for col in numeric_features:
    Q1 = df_cleaned[col].quantile(0.25)
    Q3 = df_cleaned[col].quantile(0.75)
    IQR = Q3 - Q1
    df_cleaned[col] = df_cleaned[col].clip(lower=Q1 - 1.5*IQR, upper=Q3 + 1.5*IQR)
```

- **Encoding of Categorical Features:** Since 'Credit_Mix' has a specific order (Bad is worse than Good), we used Label Encoding to map them to -1, 0, and 1. This helps the model understand the ranking. For other categories like Occupation, we used One-Hot Encoding

```
# 5. Encoding
# Label Encoding
df_cleaned = df_cleaned[df_cleaned['Credit_Mix'] != '_']
df_cleaned = df_cleaned[df_cleaned['Payment_Behaviour'] != '!@9#%8']

if 'Payment_of_Min_Amount' in df_cleaned.columns:
    df_cleaned['Payment_of_Min_Amount'] = df_cleaned['Payment_of_Min_Amount'].map({'No': 0, 'Yes': 1})
if 'Credit_Mix' in df_cleaned.columns:
    df_cleaned['Credit_Mix'] = df_cleaned['Credit_Mix'].map({'Bad': -1, 'Standard': 0, 'Good': 1})

df_analysis = df_cleaned.copy()

# One-Hot Encoding
df_cleaned = pd.get_dummies(df_cleaned, columns=['Occupation', 'Payment_Behaviour', 'Month'], drop_first=True)
```

- **Feature Scaling / Normalization:** All numerical features were standardized using StandardScaler

```
# 6. Scaling
scaler = StandardScaler()
num_cols = df_cleaned.select_dtypes(include=[np.number]).columns
df_cleaned[num_cols] = scaler.fit_transform(df_cleaned[num_cols])
```

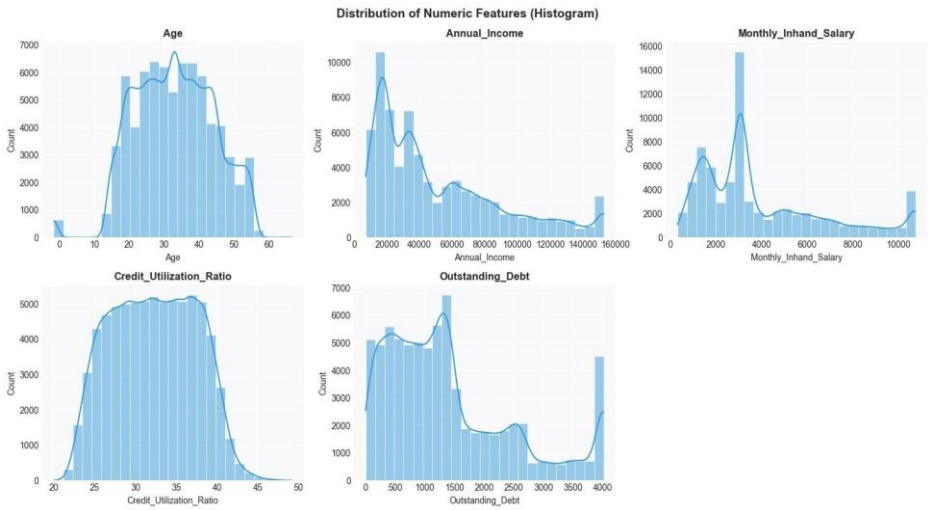
- **Initial Feature Selection:** The initial dataset was refined by selecting 24 relevant features after removing identifiers (ID, Name, etc.) and redundant variables. This initial feature selection process ensured that the models focus on the most impactful financial indicators, such as Outstanding_Debt and Interest_Rate, while reducing computational complexity.

Exploratory Data Analysis (EDA)

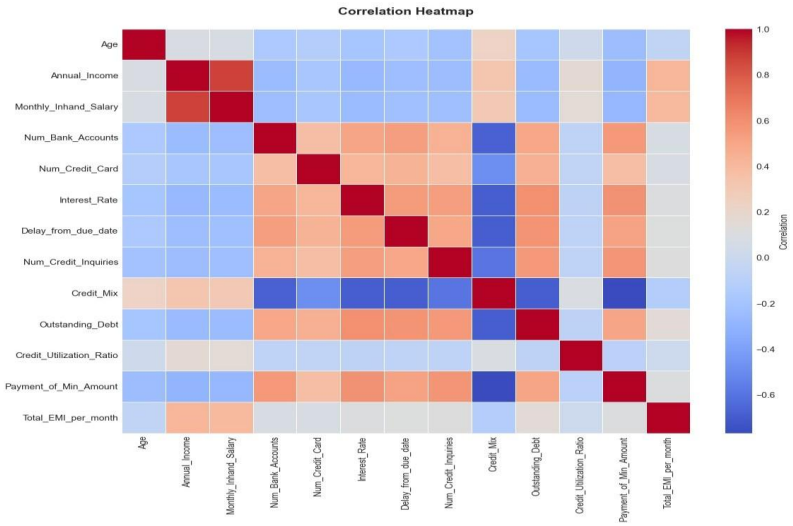
- **Statistical summary:** This subsection provides a numerical overview of the dataset after the data cleaning and pre-processing steps (handling missing values, removing duplicates, and outlier clipping).

  • Standard: ~53.2% (Majority class)
  • Poor: ~29.0%
  • Good: ~17.8% (Minority class)

- **Distribution plots:** This visualization helps identify the underlying frequency distribution of financial behaviors. For instance, we observed the right-skewed nature of income and the specific distribution of credit utilization among users.



- **Correlation heatmap:** This allows us to detect multicollinearity and identify which features are most strongly related to each other. For example, we examined the relationship between Outstanding_Debt and Interest_Rate.



- **Target variable analysis:**

```
Train: (58967, 24), Test: (14742, 24)
Target Classes: ['Good' 'Poor' 'Standard']
Encoded as: [0, 1, 2]
Class distribution:
2    0.532450
1    0.289806
0    0.177743
```

NON-ENSEMBLE MODELING

Logistic Regression: A linear baseline model used to establish a benchmark performance.

Decision Tree: A non-parametric supervised learning method used for classification. It creates a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

KNN: A simple, instance-based learning algorithm that classifies a data point based on the majority class of its 'k' nearest neighbors in the feature space.

Cross-Validation Setup

- CV technique: Stratified K-Fold Cross-Validation
- Number of folds: 5
- Evaluation metrics: Accuracy, Precision (Weighted), Recall (Weighted), F1-Score (Weighted).

Non-ensemble Models Results

Models	Evaluation Metrics **(Classification)**			
	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.6433	0.6452	0.6433	0.6410
Decision Tree	0.7123	0.7142	0.7123	0.7129
KNN	0.6924	0.6957	0.6924	0.6937

Results and Discussion

The Decision Tree achieved the best performance with an F1-Score of 0.7129, proving effective at capturing the non-linear decision boundaries inherent in credit risk data. In contrast, Logistic Regression performed the worst (F1-Score: 0.6410), as its linear nature failed to model the complex interactions between financial features. KNN showed moderate results (0.6937) but could not match the generalization capability of the rule-based tree model.

ENSEMBLE LEARNING MODELING, TUNING & COMPARISON

Ensemble Models (Write a description about each models)

- **Random Forest:** A bagging ensemble that builds multiple decision trees and merges them to reduce variance and improve accuracy.
- **Gradient Boosting:** A boosting technique that builds trees sequentially, with each new tree correcting errors made by the previous ones.
- **XGBoost:** An optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable.

Hyperparameter Tuning

Tuning Strategy

- Method used: RandomizedSearchCV
- Cross-validation folds: 3
- Scoring metric: F1-Score (weighted)

Hyperparameter Details

Model	Hyperparameters Tuned	Best Values
Random Forest	n_estimators, max_depth, min_samples_split	n_estimators: 300, min_samples_split: 2, max_depth: None
Gradient Boosting	n_estimators, learning_rate, max_depth	n_estimators: 200, learning_rate: 0.15, max_depth: 5
XGBoost	n_estimators, learning_rate, max_depth	n_estimators: 200, learning_rate: 0.15, max_depth: 7

Ensemble Learning Models Performance Table (Before Hyperparameter tuning)

Models	Evaluation Metrics **(Classification)**			
	Accuracy	Precision	Recall	F1-Score
Random Forest	0.7877	0.7886	0.7877	0.7881
Gradient Boosting	0.7129	0.7182	0.7129	0.7141
XGBoost	0.7397	0.7414	0.7397	0.7402

Results and Discussion

Performance Analysis: Ensemble models significantly outperformed all non-ensemble baselines even without optimization. Random Forest achieved the highest initial F1-Score of 0.7881, showcasing the effectiveness of bagging in handling the complexities of financial data.

Comparison:

- **Random Forest vs. Boosting:** Random Forest provided better out-of-the-box stability than XGBoost (0.7402) and Gradient Boosting (0.7141). Boosting models are more sensitive to learning rates and required further tuning to reach their potential.

- **The Ensemble Advantage:** Moving from a single Decision Tree (F1: 0.7129) to a Random Forest (F1: 0.7881) provided a substantial jump in predictive power, proving that aggregating multiple trees effectively mitigates noise in credit scoring.

Ensemble Learning Models Performance Table (After Hyperparameter tuning)

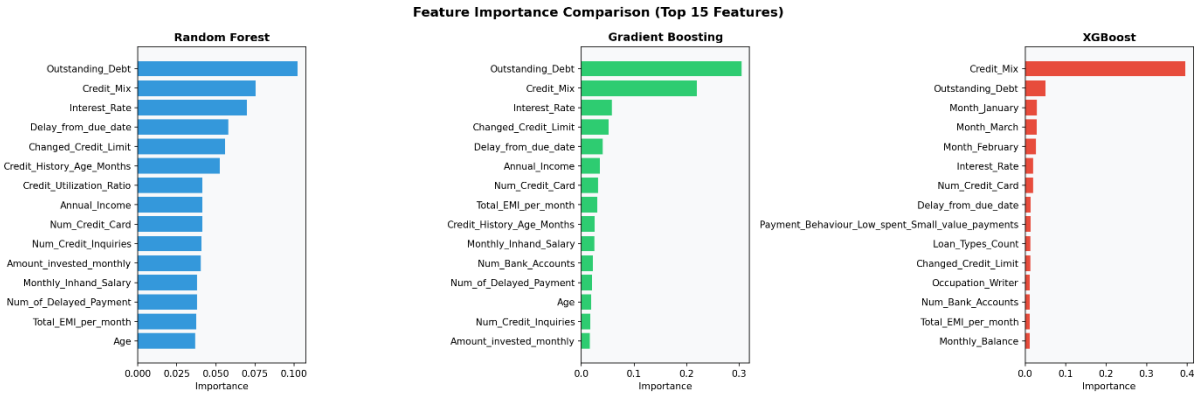
Models	Evaluation Metrics **(Classification)**			
	Accuracy	Precision	Recall	F1-Score
Random Forest	0.7889	0.7898	0.7889	0.7892
Gradient Boosting	0.7538	0.7545	0.7538	0.7540
XGBoost	0.7724	0.7729	0.7724	0.7726

Results and Discussion

Hyperparameter tuning improved all ensemble models. Gradient Boosting saw the biggest jump (F1 increased from 0.71 to 0.75). However, Random Forest remained the dominant model even after tuning, achieving the highest F1-score of 0.7892. XGBoost also performed strongly (0.7726) but could not surpass Random Forest.

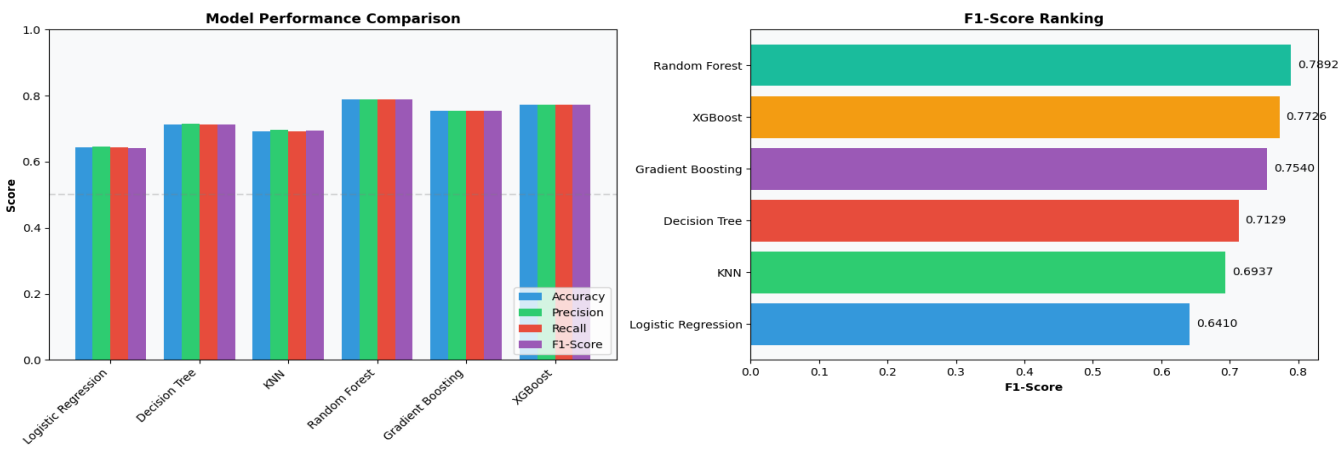
Feature Selection Analysis (Ensemble Learning Models)

Show the graph of the feature importance by the Ensemble Learning Models



Overall Results and Discussion

Comparing all the 6 models



Results and Discussion (Ensemble Models - After Tuning)

Performance Analysis: Hyperparameter tuning improved all ensemble models, particularly boosting algorithms. The Tuned Random Forest remained the top performer with an F1-Score of 0.7892, followed by XGBoost at 0.7726.

Key Findings:

- **Optimization Impact:** Gradient Boosting showed the largest gain, with its F1-Score increasing from 0.7141 to 0.7540 after tuning.
- **Efficiency:** While XGBoost offered faster training times, Random Forest provided the most robust predictive power for this dataset.

Conclusion

- Final best-performing model is Tuned Random Forest classifier.
- Random Forest achieved the highest metrics across the board: Accuracy (78.89%) and F1-Score (78.92%). It outperformed the baseline (Logistic Regression) by a massive 23.13% margin. It also showed high stability with low variance during cross-validation. While XGBoost was faster, Random Forest provided better raw predictive power for this specific dataset.

Future Work and Improvements

- **Handling Class Imbalance:** Apply SMOTE to balance the dataset and improve the recall of the minority "Good" credit class.
- **Advanced Algorithms:** Explore LightGBM or CatBoost for faster training and potentially higher accuracy on categorical data.
- **Deep Learning:** Implement Neural Networks (ANN) to capture more complex, high-dimensional patterns in financial behaviors.
- **Real-time Deployment:** Integrate the model into a web API using Streamlit or Flask for live credit score predictions.

Tools & Technologies Used

- Python 3.x – Primary programming language.
- Pandas & NumPy – Data manipulation and numerical operations.
- Scikit-learn – Machine learning models, preprocessing, and evaluation metrics.
- XGBoost – Optimized gradient boosting implementation.
- Matplotlib & Seaborn – Data visualization and plotting.
- Jupyter Notebook / Google Colab – Interactive development environment.

References

- [1] Rohan Paris, "Credit Score Classification Dataset," Kaggle, 2023. https://www.kaggle.com/datasets/parisrohan/credit-score-classification

- [2] Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, 2011. https://scikit-learn.org/stable/

- [3] XGBoost: A Scalable Tree Boosting System, Chen & Guestrin, KDD 2016. https://xgboost.readthedocs.io/

- Seaborn: Statistical Data Visualization, Michael Waskom. https://seaborn.pydata.org/